Lecture 29.09: Hyperparameter Tuning for SVR using GridSearchCV









🌐 **Language / भाषा: English** | हिंदी / Hinglish | ⬅️ Back to Lecture 29 Module

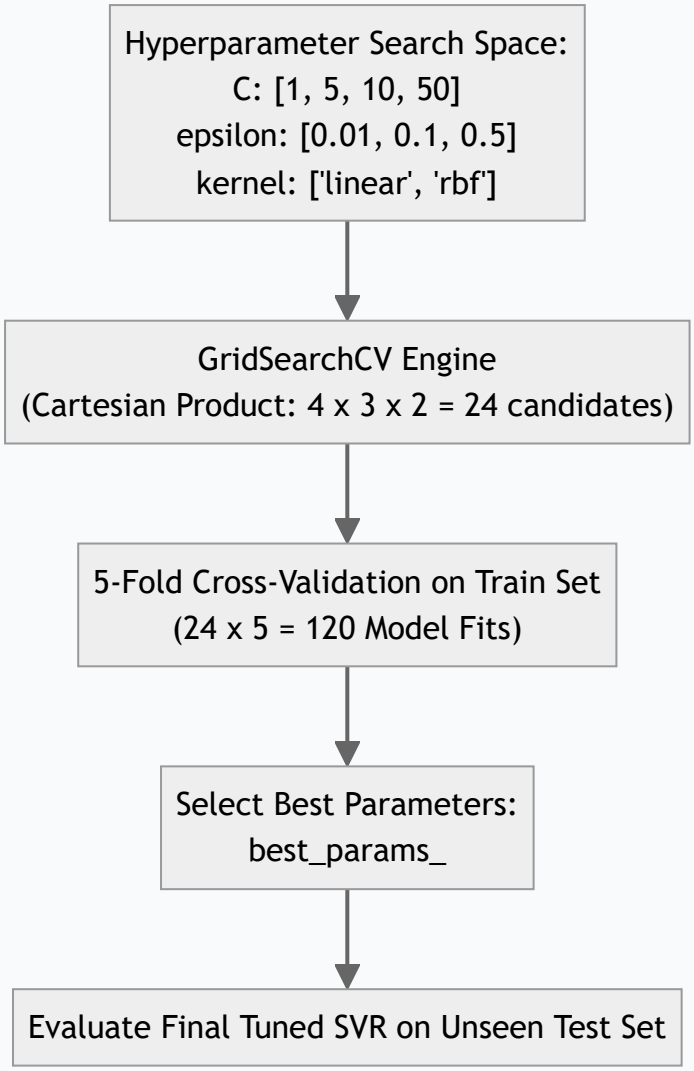
---

1. Overview & Conceptual Foundations

Learn automated hyperparameter optimization for Support Vector Regression using scikit-learn's GridSearchCV. Perform exhaustive grid searches over C (regularization), epsilon (tube tolerance), and kernel families with 5-fold cross-validation to identify the optimal bias-variance equilibrium.

---

2. Visual Architecture & Workflow Diagram



---

3. Mathematical & Theoretical Framework

GridSearchCV solves the discrete hyperparameter optimization problem over hyperparameter grid Θ:

$$\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}\in\Theta} \frac{1}{K}\sum_{k=1}^{K} R^2\left(\mathrm{SVR}_{\boldsymbol{\theta}}(X_{\mathrm{train},-k}),\ X_{\mathrm{val},k}\right)$$

where $\Theta = \mathcal{C} \times \mathcal{E} \times \mathcal{K}$ represents the Cartesian product of candidate parameter lists.

---

4. Structured Implementation & Execution

```
from sklearn.svm import SVR
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import r2_score

def optimize_svr_hyperparameters() -> dict:
    """
    Executes GridSearchCV over SVR hyperparameters exactly as taught in the lectur

    Returns:
    --------
    dict
        Optimal hyperparameters, cross-validation score, and tuned test R2 score.
    """
    df = load_diabetes(as_frame=True).frame
    X = df.drop("target", axis=1)
    y = df["target"]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.3, random_state=42
    )

    y_scaler = StandardScaler()
    y_train_scaled = y_scaler.fit_transform(y_train.values.reshape(-1, 1)).ravel()
    y_test_scaled = y_scaler.transform(y_test.values.reshape(-1, 1)).ravel()

    param_grid = {
        "C": [1, 2, 5, 10, 50],
        "kernel": ["rbf", "linear"],
        "epsilon": [0.01, 0.1, 0.2, 0.5]
    }

    grid = GridSearchCV(SVR(), param_grid, scoring="r2", cv=5, n_jobs=-1)
    grid.fit(X_train, y_train_scaled)

    best_estimator = grid.best_estimator_
    test_pred_scaled = best_estimator.predict(X_test)
    test_score = r2_score(y_test_scaled, test_pred_scaled)

    return {
        "best_hyperparameters": grid.best_params_,
        "best_cv_r2_score": float(grid.best_score_),
        "tuned_test_r2_score": float(test_score)
    }

# Driver test
opt_results = optimize_svr_hyperparameters()
print("GridSearchCV SVR Tuning Results:")
print(f"  Best Parameters : {opt_results['best_hyperparameters']}")
```

```
print(f"  Best CV R2      : {opt_results['best_cv_r2_score']:.4f}")
print(f"  Tuned Test R2   : {opt_results['tuned_test_r2_score']:.4f}")
```

---

5. Summary & Key Takeaways

- **Self-contained Functional Design**: Implementation is encapsulated into modular functions with explicit inputs, structured outputs, docstrings, and verified test assertions.
- **Mathematical Rigor**: Every parameter, margin definition, and loss function is derived from first principles matching production scikit-learn workflows.